

Week 2 · 실습 워크북 — 화살표를 긋고, state를 뜯고, 역순으로 지운다

참고

이 문서는 실습 파트(43분)입니다. 개념은 **개념워크북** (같은 **lecture/** 폴더) 을 먼저 보세요. **[실습]** 은 직접 치고, **[확인]** 은 눈으로 대조하고, **[함정]** 은 미리 피하고, **[관찰 🕒]** 은 빈칸을 채웁니다. **(심화 ⭐)** 는 먼저 끝낸 사람만 합니다.

오늘 끝나면: VPC부터 EC2까지 7개를 배포하고, **tfstate** 안에서 내가 적지 않은 의존성 그래프를 찾아내고, 역순으로 전부 지울 수 있다.

오늘 만들 것

VPC + public subnet + IGW + route table + association + security group + EC2 t3.micro. 리소스 7개, 참조(화살표) 11개.

⚠ 절대 금지

오늘부터 진짜로 돈이 나갑니다. EC2는 프리티어가 남아 있으면 \$0이지만, 프리티어가 끝난 계정은 시간당 과금됩니다 (EC2 \$0.0130 + 퍼블릭 IPv4 \$0.005 + EBS 8GiB ≈ 시간당 \$0.019, 약 27원).

43분 실습 = 약 20원. destroy를 잊고 한 달 방치 = 약 \$13.9(약 2만원). 차이는 **terraform destroy** 한 줄입니다.

NAT Gateway와 Elastic IP는 절대 만들지 마세요. 프리티어가 없고 시간당 과금됩니다.

! 꼭 기억

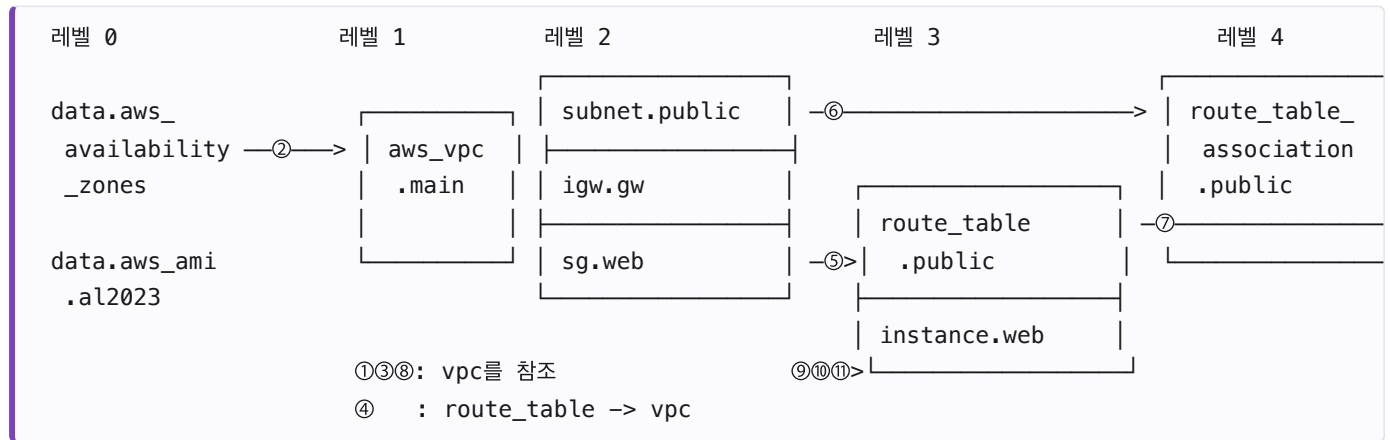
작업 폴더는 이 리포의 **practice/** 하나뿐입니다. 새 폴더를 만들지 마세요.

versions.tf · providers.tf · variables.tf 는 준비돼 있고, 여러분은 **network.tf · compute.tf · outputs.tf** 의 **# TODO** 만 채웁니다.

파일	무슨 일을 하나	오늘 손대나
versions.tf	Terraform·provider 버전 고정	읽기만
providers.tf	리전 + default_tags	읽기만
variables.tf	입력 변수 선언 + validation	읽기만
network.tf	VPC·subnet·IGW·route table·association	채운다 (TODO 4개)
compute.tf	AMI 데이터 소스·SG·EC2	채운다 (TODO 2개)
outputs.tf	출력값 6개	채운다 (TODO 2개)

오늘의 지도 — 화살표 지도 (Arrow Map)

Week1의 지도는 명령의 흐름(Scope → Author → Initialize → Plan → Apply)이었습니다. 오늘의 지도는 리소스 사이의 흐름입니다. 이 지도를 한 세션에 세 번 씩입니다 — Block A에서 **긋고**, Block B에서 apply 로그와 **대조하고**, Block C에서 뒤집습니다.



화살표 번호와 뜻은 개념워크북 1번 표와 같습니다. 이 그림만 "만들어지는 순서" 방향(왼쪽 → 오른쪽)으로 그렸습니다. 실제 참조는 그 반대 방향입니다 — `subnet` 이 `vpc` 를 참조하므로, 코드와 `terraform graph`에서는 `aws_subnet.public -> aws_vpc.main` (= "subnet이 vpc를 기다린다")로 나옵니다.

스텝	여기서 긋는 화살표	하는 일
A-3	① ②	subnet 작성
A-4	③ ④ ⑤ ⑥ ⑦	IGW · route table · association 작성
A-5	—	plan — Plan: 5 to add
A-6	—	네트워크 5개 apply
B-1	⑧ ⑨ ⑩ ⑪	SG · EC2 작성
B-2	—	plan — Plan: 2 to add
B-3	—	컴퓨터 2개 apply
C-1 · C-2	전부 역방향	7개 destroy (앞사귀부터)

참고

지금 몇 번 화살표를 긋는 중이지만 알면 길을 잃지 않습니다. 그리고 화살표를 다 그으면 순서는 적지 않아도 됩니다. 그게 오늘의 전부입니다.

팁

레벨 4가 EC2가 아닙니다. 인자가 참조 두 개뿐인 `route_table_association` 이 가장 마지막입니다. 왜 그런지는 B-3에서 로그로 확인합니다.

오늘 세 번 나오는 숫자를 미리 못 박아 둡니다. 이것과 다르면 뭔가 잘못된 것입니다.

지점	숫자
A-5 plan (네트워크만)	Plan: 5 to add, 0 to change, 0 to destroy.
A-6 후 <code>state list</code>	7줄 (리소스 5 + 데이터 소스 2)
B-2 plan (EC2 추가)	Plan: 2 to add, 0 to change, 0 to destroy.
B-4 <code>state list</code>	9줄 (리소스 7 + 데이터 소스 2)
C-1 destroy plan	Plan: 0 to add, 0 to change, 7 to destroy.
C-2 완료	Destroy complete! Resources: 7 destroyed.

Block A. 워크스루 (12분) — 강사와 함께

화살표 ① ~ ⑦ 을 갖고 네트워크 5개만 먼저 apply

[실습] A-1. 지난주와 달라진 폴더 — 파일이 왜 쪼개졌나

```
cd practice
ls
terraform version          # 1.9.0 이상
aws sts get-caller-identity # 자격증명이 살아 있는지
```

Week1은 `main.tf` 하나였는데 오늘은 `network.tf` 와 `compute.tf` 로 나뉘어 있습니다.

Terraform은 파일 경계를 모릅니다. 현재 폴더의 `.tf` 를 전부 읽어 하나로 합칩니다(Week1 A-2). 그래서 쪼개는 기준은 문법이 아니라 사람이 찾는 단위입니다. `network.tf` 에 있는 `aws_vpc.main` 을 `compute.tf` 에서 그냥 참조할 수 있습니다 — 파일이 달라도 같은 코드입니다.

⚠ 주의

터미널은 하나만 씁니다. 두 개에서 동시에 명령을 돌리면 `Error acquiring the state lock` 이 납니다.

[실습] A-2. 내 값 채우기 — `project_name` · `my_ip`

```
cp example.tfvars terraform.tfvars
curl -4 ifconfig.me          # 내 공인 IPv4 한 줄이 나온다
```

`terraform.tfvars` 를 열어 두 값을 바꿉니다.

```
region      = "ap-northeast-2"
project_name = "boaz26-w2-{본인-github-id}" # 예: boaz26-w2-kdh1834
my_ip       = "1.2.3.4"                     # curl -4 결과. /32 를 붙이지 말 것
instance_type = "t3.micro"
```

! 꼭 기억

`my_ip` 에 `/32` 를 붙이지 마세요. 코드가 `"${var.my_ip}/32"` 로 조립합니다. `1.2.3.4/32` 를 넣으면 `1.2.3.4/32/32` 가 됩니다.

`variables.tf` 가 이걸 막아줍니다. 값이 `CHANGE-ME` 거나 형식이 틀리면 `plan`이 아예 시작되지 않습니다.

Error: Invalid value for variable

my_ip는 1.2.3.4 처럼 순수 IPv4여야 합니다. /32나 CIDR을 넣지 마세요.

이건 좋은 실패입니다. 이 검사가 없으면 SG를 만드는 순간(apply 중간)에 `InvalidParameterValue` 로 죽습니다. 그때는 VPC·subnet·IGW가 이미 만들어져 있어서 가장 당황하는 지점입니다.



팁

curl 에 -4 를 꼭 붙이세요. 안 붙이면 IPv6(2001:...)가 나올 수 있고, 그건 /32 를 붙일 수 없는 값입니다.

! 꼭 기억

CP1 — Discord에 `project_name` 한 줄만 올려주세요. 오타를 apply 전에 걸러냅니다.

🚫 절대 금지

`my_ip` 값은 Discord에 올리지 마세요. 본인 공인 IP입니다. `curl -4 ifconfig.me` 가 IPv4 한 줄을 받았는지
여부만(OK / 실패) 보고하세요.

[실습] A-3. 첫 화살표 ① ② — subnet

`network.tf` 의 첫 TODO입니다. `data "aws_availability_zones"` 와 `aws_vpc.main` 은 이미 채워져 있습니다.

```
resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id                # 화살표 ①
  cidr_block        = "10.0.1.0/24"
  availability_zone = data.aws_availability_zones.available.names[0] # 화살표 ②

  map_public_ip_on_launch = true

  tags = { Name = "${var.project_name}-public" }
}
```

읽는 규칙은 하나입니다. 왼쪽은 인자 이름(provider가 정함), 오른쪽은 참조(내가 씀).

```
vpc_id = aws_vpc.main.id
  ^      ^
  |      +---- 오른쪽: 내가 붙인 주소(aws_vpc.main) + 속성(.id)
+----- 왼쪽: aws_subnet 이 요구하는 인자 이름
```

여기서 세 가지를 짚습니다.

- ① `availability_zone` 을 하드코딩하지 않습니다. `"ap-northeast-2a"` 라고 적으면 안 되는 이유는 두 개입니다.
 - AZ 이름은 계정마다 다른 물리 AZ에 매핑됩니다. 오래된 계정은 특정 AZ에 서브넷을 못 만들 수도 있습니다.
 - AZ × 인스턴스 타입 조합이 있습니다. `t3.micro` 는 서울 4개 AZ 전부 가능하지만 `t2.micro` 는 2b·2d에 없습니다.
- ② `map_public_ip_on_launch = true` 한 줄이 퍼블릭 IP의 정체입니다. 이 서브넷에서 뜨는 인스턴스는 퍼블릭 IPv4를 자동으로 받습니다. 그래서 우리는 Elastic IP를 만들지 않습니다. EIP는 유휴 상태에서도 시간당 과금됩니다.
- ③ 인터넷에 닿으려면 세 가지가 다 필요합니다. 하나만 빠져도 조용히 안 됩니다.

- ① 주소가 있는가 : `map_public_ip_on_launch = true` <- A-3
- ② 나가는 길이 있는가 : `IGW + route table 0.0.0.0/0 + association` <- A-4
- ③ 방화벽이 허용하는가: `SG egress 전체 허용` <- B-1

[실습] A-4. 화살표 ③ ④ ⑤ ⑥ ⑦ — IGW · route table · association

`network.tf` 의 남은 TODO 3개를 채웁니다.

```
resource "aws_internet_gateway" "gw" {
  vpc_id = aws_vpc.main.id # 화살표 ③
  tags   = { Name = "${var.project_name}-igw" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id # 화살표 ④

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.gw.id # 화살표 ⑤
  }

  tags = { Name = "${var.project_name}-rt-public" }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id      # 화살표 ⑥
  route_table_id = aws_route_table.public.id # 화살표 ⑦
}
```

그리고 `outputs.tf` 의 첫 TODO도 지금 채웁니다.

```
output "vpc_id" {
  description = "생성된 VPC ID"
  value       = aws_vpc.main.id
}

output "subnet_id" {
  description = "퍼블릭 서브넷 ID"
  value       = aws_subnet.public.id
}

output "subnet_az" {
  description = "서브넷이 놓인 AZ - data.aws_availability_zones 가 골라준 값"
  value       = aws_subnet.public.availability_zone
}
```

여기서 볼 것이 두 개입니다.

route 는 리소스가 아니라 인자입니다. `resource "aws_route_table"` 안에 중괄호로 들어간 `route { }` 는 별개의 리소스가 아니라 **라우트 테이블의 속성**입니다. 겉모습이 `resource` 블록과 비슷해서 헷갈리기 쉽습니다. 이렇게 안에 들어가는 것을 **인라인 블록**이라고 부릅니다.

association 은 "연결 자체가 리소스"입니다. AWS 콘솔에는 이걸 위한 독립 화면이 없습니다(라우트 테이블 화면의 탭으로 보입니다). 그런데 Terraform에서는 리소스 하나이고, 인자가 참조 두 개뿐입니다. 그래서 화살표 지도에서 **가장 마지막 레벨**입니다.

[실습] A-5. `fmt` → `validate` → `plan` — 화살표를 `plan`에서 읽기

```
terraform fmt      # 정렬 교정. 고친 파일 이름만 출력
terraform init     # provider 다운로드 (예습에서 이미 했으면 몇 초)
terraform validate  # 문법·정합성 검사
terraform plan
```

[확인] `plan` 맨 위 — 데이터 소스가 먼저 읽힌다

```
data.aws_ami.al2023: Reading...
data.aws_availability_zones.available: Reading...
data.aws_availability_zones.available: Read complete after 1s [id=ap-northeast-2]
data.aws_ami.al2023: Read complete after 1s [id=ami-04bb4ddfc0e51bb5e]
```

데이터 소스 2개가 `plan` 단계에서 이미 읽혔습니다. `compute.tf` 의 SG·EC2는 아직 주석인데도 `data "aws_ami"` 블록은 살아 있으니 읽힙니다. Week1 A-8에서 본 그 두 줄이 오늘은 두 개입니다.

[확인] `subnet` 블록 — 확정된 값과 아직 모르는 값

```
# aws_subnet.public will be created
+ resource "aws_subnet" "public" {
  + availability_zone           = "ap-northeast-2a"
  + availability_zone_id       = (known after apply)
  + cidr_block                 = "10.0.1.0/24"
  + id                        = (known after apply)
  + map_public_ip_on_launch    = true
  + tags                      = {
    + "Name" = "boaz26-w2-kdh1834-public"
  }
  + tags_all                  = {
    + "ManagedBy" = "terraform"
    + "Name"       = "boaz26-w2-kdh1834-public"
    + "Project"    = "boaz26-w2-kdh1834"
    + "Study"      = "boaz-terraform-26"
    + "Week"       = "2"
  }
  + vpc_id                   = (known after apply)
}
```

세 줄을 나란히 보세요. 이게 오늘의 핵심입니다.

줄	값	왜
<code>cidr_block</code>	<code>"10.0.1.0/24"</code>	내가 직접 적은 값이라 plan 때 이미 안다
<code>availability_zone</code>	<code>"ap-northeast-2a"</code>	데이터 소스가 읽어온 값이라 plan 때 이미 안다
<code>vpc_id</code>	<code>(known after apply)</code>	VPC를 만들어야 정해지는 값 → 화살표 ①의 정체

`(known after apply)` 가 화살표의 증거입니다. "값을 아직 모른다"가 곧 "그것보다 나중에 만들어야 한다"입니다.

그리고 `tags` 와 `tags_all` 을 비교하세요. `tags` 는 내가 쓴 것(Name 1개), `tags_all` 은 `providers.tf` 의 `default_tags` 와 병합된 최종 결과(5개) 입니다. 내가 이 리소스에 쓰지 않은 태그 4개 (`Project` · `Study` · `Week` · `ManagedBy`)가 자동으로 붙었습니다.

```
Plan: 5 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ subnet_az = "ap-northeast-2a"
+ subnet_id = (known after apply)
+ vpc_id    = (known after apply)
```

! 꼭 기억

Plan: 5 to add 가 아니면 멈추세요. 4면 TODO가 덜 채워졌고, 7이면 `compute.tf` 의 TODO를 미리 채운 것입니다(그것도 괜찮지만 A-6의 로그 관찰이 흐려집니다).

[함정] plan이 멈추거나 이상할 때

증상	원인	해결
<code>my_ip</code> 는 <code>1.2.3.4</code> 처럼... 에러	<code>terraform.tfvars</code> 가 <code>CHANGE-ME</code> 그대로	A-2를 다시
<code>var.project_name</code> 을 입력하라며 멈춤	<code>terraform.tfvars</code> 를 안 만들	Ctrl+C로 빠져나오고 파일부터 만들 것. 아무거나 치면 그 값으로 apply된다
<code>No changes.</code>	TODO가 아직 주석 상태	주석을 풀고 채우기
<code>Reference to undeclared resource</code>	주소 오타 (<code>aws_vpc.main</code> ↔ <code>aws_vpc.mian</code>)	<code>validate</code> 가 줄 번호를 알려준다
<code>Inconsistent dependency lock file</code>	<code>init</code> 을 안 함	<code>terraform init</code>
<code>Your query returned no results</code>	AMI 필터가 아무것도 못 잡음	리전이 <code>ap-northeast-2</code> 인지 확인

[실습] A-6. 네트워크만 먼저 apply — 순서를 로그로 보기

```
terraform apply # yes
```

[확인] apply 로그의 초 단위가 곧 레벨이다

```
aws_vpc.main: Creating...
aws_vpc.main: Creation complete after 2s [id=vpc-0f9e8d7c6b5a4321f]
aws_subnet.public: Creating...
aws_internet_gateway.gw: Creating...
aws_subnet.public: Creation complete after 1s [id=subnet-0a1b2c3d4e5f67890]
aws_internet_gateway.gw: Creation complete after 1s [id=igw-0123456789abcdef0]
aws_route_table.public: Creating...
aws_route_table.public: Creation complete after 1s [id=rtb-0fedcba9876543210]
aws_route_table_association.public: Creating...
aws_route_table_association.public: Creation complete after 0s [id=rtbassoc-0aaa...]

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

Outputs:

subnet_az = "ap-northeast-2a"
subnet_id = "subnet-0a1b2c3d4e5f67890"
vpc_id    = "vpc-0f9e8d7c6b5a4321f"
```

읽을 곳이 세 군데입니다.

① **aws_vpc.main** 이 혼자 먼저입니다. 아무것도 참조하지 않는 유일한 리소스라 여기가 병목입니다. ② **subnet** 과 **igw** 의 **Creating...** 이 붙어서 뜹니다. 둘은 서로를 참조하지 않으므로 동시에 시작합니다(레벨 2). ③ **association** 이 마지막입니다. subnet(⑥)과 route table(⑦) 둘 다 필요하니까요.

나는 어디에도 순서를 적지 않았습시다. 화살표 ①~⑦만 그었을 뿐입니다.

```
terraform state list
```

```
data.aws_ami.al2023
data.aws_availability_zones.available
aws_internet_gateway.gw
aws_route_table.public
aws_route_table_association.public
aws_subnet.public
aws_vpc.main
```

7줄입니다. 리소스 5 + 데이터 소스 2. 데이터 소스는 만든 게 없는데도 state에 적힙니다 — 다음 plan에서 "그때 값과 지금 값이 같은가"를 비교해야 하기 때문입니다.

[관찰 🍷] A-6 기록

- `Creating...` 이 같은 순간에 뜬 리소스 이름 전부: _____
- `aws_route_table.public` 이 `aws_internet_gateway.gw` 보다 늦게 시작한 이유를 **화살표 번호**로: _____
- 가장 마지막에 `Creation complete` 가 뜬 리소스: _____ / 화살표 지도의 예측과 같은가: _____
- `state list` 줄 수: _____ / 그중 실제 AWS 리소스: _____
- `subnet_az` 값: _____ (옆 사람과 같은가? 다르면 왜?)

[실습] A-7. terraform graph — 그래프를 텍스트로 보기 (선택)

```
terraform graph | grep '\->' | sed 's/"//g; s/;/;' | sort
```

```
aws_internet_gateway.gw -> aws_vpc.main
aws_route_table_association.public -> aws_route_table.public
aws_route_table_association.public -> aws_subnet.public
aws_route_table.public -> aws_internet_gateway.gw
aws_subnet.public -> aws_vpc.main
aws_subnet.public -> data.aws_availability_zones.available
```

A → B 는 "A가 B를 기다린다"는 뜻입니다. 화살표 지도의 방향을 뒤집은 것이고, 이쪽이 코드에 적힌 참조 방향입니다.

🔗 참고

왜 `aws_route_table.public -> aws_vpc.main` (화살표 ④)이 안 보이나? `terraform graph` 의 기본 출력은 요약이라서, 다른 경로로 이미 함축되는 간선을 생략합니다. route table은 IGW를 기다리고 IGW는 VPC를 기다리므로 "route table → VPC"는 자동으로 보장됩니다. 코드에는 ④가 분명히 있지만 그림에는 없는 것이고, **B-4에서 볼 state의 dependencies** 배열에는 다시 나타납니다.

💡 팁

`dot` (Graphviz)을 설치하지 마세요 — 세션 시간이 없습니다. 그림이 꼭 보고 싶으면 `terraform graph` 출력을 통째로 복사해 <https://dreampuf.github.io/GraphvizOnline/> 에 붙여넣으면 됩니다.

Block B. 각자 (20분) — EC2 · state 해부 · drift

화살표 ⑧ ⑨ ⑩ ⑪ 을 굿고, 장부를 뜯는다

! 꼭 기억

B-1 ~ B-4가 L1(필수)입니다. B-5부터는 심화입니다. 강사가 "8분 남았습니다"라고 공지하면 어디에 있든 Block C로 이동하세요. 못 끝낸 사람도 **destroy**는 합니다 — 절반만 만들어진 것도 과금됩니다.

[실습] B-1. 화살표 ⑧ ⑨ ⑩ ⑪ — SG + EC2

`compute.tf` 의 TODO 2개를 채웁니다. `data "aws_ami"` 는 이미 채워져 있습니다.

```

resource "aws_security_group" "web" {
  name           = "${var.project_name}-web-sg"
  description    = "boaz w2 lab: SSH from my IP only"
  vpc_id         = aws_vpc.main.id # 화살표 ⑧

  ingress {
    description = "SSH from my IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["${var.my_ip}/32"]
  }

  egress {
    description = "all outbound"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = { Name = "${var.project_name}-web-sg" }
}

resource "aws_instance" "web" {
  ami                = data.aws_ami.al2023.id      # 화살표 ⑪
  instance_type      = var.instance_type
  subnet_id          = aws_subnet.public.id        # 화살표 ⑨
  vpc_security_group_ids = [aws_security_group.web.id] # 화살표 ⑩

  root_block_device {
    volume_type      = "gp3"
    volume_size      = 8
    delete_on_termination = true
  }

  metadata_options {
    http_tokens = "required"
  }

  tags = { Name = "${var.project_name}-web" }
}

```

`outputs.tf` 의 두 번째 TODO도 채웁니다.

```

output "ami_id" {
  description = "data.aws_ami 가 찾아낸 AL2023 AMI ID"
  value       = data.aws_ami.al2023.id
}
output "instance_id" {
  description = "EC2 인스턴스 ID (destroy 전에 기록해 두면 증빙이 된다)"
  value       = aws_instance.web.id
}
output "instance_public_ip" {
  description = "자동 할당된 퍼블릭 IPv4. EIP가 아니므로 stop/start 시 바뀐다. Discord에 올리지 말 것"
  value       = aws_instance.web.public_ip
}

```

여기서 짚을 것 다섯 개.

① `vpc_id = aws_vpc.main.id` 를 SG에 빼먹으면 안 됩니다. 빼면 SG가 기본 VPC에 만들어지고, EC2 생성에서 이렇게 죽습니다.

```

Error: creating EC2 Instance: InvalidParameterValue:
  Security group sg-xxx and subnet subnet-yyy belong to different networks.

```

② `vpc_security_group_ids` 는 리스트입니다. SG는 한 인스턴스에 여러 개 붙을 수 있어서 하나만 붙여도 `[ ]` 가 필요합니다.

③ `ami = data.aws_ami.al2023.id` 가 오늘 유일한 `data.` 참조입니다. `resource` 는 `aws_subnet.public` , `data` 는 `data.aws_ami.al2023` — 앞에 `data.` 가 붙습니다.

④ `cidr_blocks = ["0.0.0.0/0"]` 로 22번을 열지 마세요. 몇 분 안에 스캔 트래픽이 붙습니다. 내 IP /32 만.

⑤ `associate_public_ip_address` 를 쓰지 마세요. 퍼블릭 IP는 서브넷의 `map_public_ip_on_launch` 하나로만 통제합니다. 두 곳에 진실이 갈리고, 나중에 이 값을 바꾸면 인스턴스가 통째로 재생성(-/+) 됩니다.

🚫 절대 금지

키페어를 만들지 마세요. 오늘은 SSH 접속을 하지 않습니다. 코드로 키를 만들면 비밀키가 tfstate에 평문 저장되고 `.pem` 파일이 커밋 사고를 냅니다. SG에 22번 규칙을 남기는 이유는 문법 학습 + B-5 drift 실습 교보재 + 최소 권한 습관입니다. 오늘의 성공 기준은 접속이 아니라 `instance_state = running` + 퍼블릭 IP 할당입니다.

[확인] B-2. 두 번째 plan — (known after apply) 가 줄었다

```

terraform fmt && terraform validate
terraform plan

```

```
# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami                = "ami-04bb4ddfc0e51bb5e"
  + instance_type      = "t3.micro"
  + private_ip         = (known after apply)
  + public_ip          = (known after apply)
  + subnet_id          = "subnet-0a1b2c3d4e5f67890"
  + vpc_security_group_ids = (known after apply)
  ...
}
```

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ ami_id              = "ami-04bb4ddfc0e51bb5e"
+ instance_id         = (known after apply)
+ instance_public_ip = (known after apply)
```

A-5와 비교하세요. `subnet_id` 가 **(known after apply)** 가 아니라 실제 값입니다. 서브넷은 이미 만들어져 **state**에 적혀 있으니 Terraform이 그 값을 압니다. 화살표 ⑨는 그대로 있지만, 가리키는 대상이 이미 존재하는 것이죠.

반면 `vpc_security_group_ids` 는 아직 **(known after apply)** 입니다 — SG를 이번에 만들 거니까요(화살표 ⑩).



팁

ami_id 값을 옆 사람과 비교해보세요. 같은 리전·같은 필터라면 같은 값이 나옵니다. 데이터 소스는 같은 질문에 같은 답을 줍니다. 다만 `most_recent = true` 라서 다음 주에 돌리면 다른 값이 나올 수 있습니다. 그리고 `ami` 는 `-/+` (교체) 속성입니다 — 코드를 안 고쳤는데 EC2가 재생성될 수 있다는 뜻입니다(B-6에서 확인).

[실습] B-3. apply — EC2가 뜬다

```
terraform apply # yes
```



주의

Still creating... [10s elapsed] 가 정상입니다. EC2는 provider가 **running** 상태까지 기다리므로 30~60초 걸립니다. 화면이 멈춘 것처럼 보여도 **Ctrl+C**를 절대 누르지 마세요.

Ctrl+C로 강제 종료하면 ① state 잠금이 남아 다음 명령이 **Error acquiring the state lock** 으로 죽고 ② **state**에 안 적힌 채 **AWS**에만 존재하는 리소스가 생길 수 있습니다. 그건 **destroy** 로도 안 지워지는 고아 리소스, 즉 아무도 안 지우는 청구서입니다.

[확인] apply 성공 신호

```
aws_security_group.web: Creating...
aws_security_group.web: Creation complete after 3s [id=sg-0abc123def456789a]
aws_instance.web: Creating...
aws_instance.web: Still creating... [10s elapsed]
aws_instance.web: Still creating... [20s elapsed]
aws_instance.web: Still creating... [30s elapsed]
aws_instance.web: Creation complete after 33s [id=i-0123456789abcdef0]
```

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

```
ami_id           = "ami-04bb4ddfc0e51bb5e"
instance_id      = "i-0123456789abcdef0"
instance_public_ip = "3.35.xxx.xxx"
subnet_az        = "ap-northeast-2a"
subnet_id        = "subnet-0a1b2c3d4e5f67890"
vpc_id           = "vpc-0f9e8d7c6b5a4321f"
```

SG가 먼저, EC2가 나중입니다. 화살표 Ⓢ 때문입니다. 그리고 EC2 하나가 나머지 6개를 합친 것보다 오래 걸립니다.

! 꼭 기억

CP2 — **Apply complete! Resources: 2 added.** 줄 + **terraform state list** 출력 전체를 **project_name** 과 함께 Discord에 텍스트로 올려주세요. (스크린샷 말고 텍스트)

🚫 절대 금지

instance_public_ip 값은 올리지 마세요. 22번이 열려 있는 서버의 주소입니다. 화면에서만 확인하고, Discord에는 **state list** 만.

[관찰 🍷] B-3 기록

- Creation complete** 순서: _____ → _____
- EC2 하나를 만드는 데 걸린 초: _____ / A-6의 네트워크 5개 전체는: _____
- 화살표 지도의 **레벨 4**는 **association**인데, 로그에서 **가장 늦게 끝난** 것은 무엇이었나: _____
- 왜 다른가 (한 문장): _____
- 퍼블릭 IP가 붙었는데 EIP를 만들지 않았습니다. 어느 인자 덕인가: _____

[실습] B-4. state 해부 — 오늘의 하이라이트

apply는 리소스를 만드는 것으로 끝나지 않습니다. **장부에 적는 것까지가 한 세트**입니다. 그 장부를 뜯습니다.

① 주소 목록

```
terraform state list
```

```
data.aws_ami.al2023
data.aws_availability_zones.available
aws_instance.web
aws_internet_gateway.gw
aws_route_table.public
aws_route_table_association.public
aws_security_group.web
aws_subnet.public
aws_vpc.main
```

9줄 = 리소스 7 + 데이터 소스 2. Plan: 5 to add + Plan: 2 to add = 7과 맞습니다.

② 하나만 파보기

```
terraform state show aws_instance.web
```

확인할 것: `id` · `private_ip` · `public_ip` · `subnet_id` · `ami` · `instance_state` · `tags` vs `tags_all`.

```
terraform state show aws_subnet.public | grep -E "availability_zone|cidr_block|map_public"
```

`availability_zone = "ap-northeast-2a"` — 데이터 소스가 골라준 값이 여기 저장돼 있습니다.

③ 파일을 직접 열기 — 최상위 키 7개

```
ls -la terraform.tfstate
jq 'keys' terraform.tfstate
jq '{version, terraform_version, serial, lineage}' terraform.tfstate
```

```
{
  "version": 4,
  "terraform_version": "1.15.8",
  "serial": 3,
  "lineage": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
}
```

키	뜻
<code>version: 4</code>	state 파일 포맷 버전. Terraform 버전이 아님
<code>serial</code>	장부가 저장된 횟수. apply마다 늘어난다 → Week3 잠금의 기반
<code>lineage</code>	이 장부의 혈통 UUID. 다른 인프라의 장부를 밀어 넣는 사고를 막는다

④ 리소스 목록과 속성

```
jq -r '.resources[] | "\(.mode) \(.type).\(.name)"' terraform.tfstate
jq '.resources[] | select(.type=="aws_instance") | .instances[0].attributes
  | {id, ami, instance_type, subnet_id, private_ip, public_ip, tags, tags_all}' terraform.tfstate
```

❗ 절대 금지

attributes 에는 우리가 쓴 것보다 훨씬 많은 값이 들어 있습니다. AWS가 정해진 기본값까지 전부입니다. 그리고 **plan**이 ~ 인지 **-/+** 인지 계산하는 근거가 바로 이 값들입니다. 전부 평문입니다.

⑤ ★ dependencies — 내가 적지 않은 그래프

```
jq -r '.resources[] | "\(.type).\(.name) <- \(.instances[0].dependencies // [] | join(",
  "))"' terraform.tfstate
```

```
aws_instance.web <- aws_security_group.web, aws_subnet.public, aws_vpc.main, data.aws_ami.al2
023, ...
aws_internet_gateway.gw <- aws_vpc.main
aws_route_table.public <- aws_internet_gateway.gw, aws_vpc.main
aws_route_table_association.public <- aws_route_table.public, aws_subnet.public, aws_vpc.main, ...
aws_security_group.web <- aws_vpc.main
aws_subnet.public <- aws_vpc.main, data.aws_availability_zones.available
aws_vpc.main <-
```

바로 옆에 이걸 띄워 나란히 비교하세요.

```
terraform graph | grep '\->' | sed 's/"//g; s/;/;/g' | sort
```

! 꼭 기억

왼쪽은 **.tf** 파일에서 계산한 그래프, 오른쪽은 state에 저장된 그래프입니다. 같습니다.

그런데 왜 state에 또 적어둘까요? **.tf** 파일을 다 지워도 **terraform destroy** 가 올바른 순서로 동작해야 하기 때문입니다. 코드가 사라지면 참조도 사라지지만, 장부의 **dependencies** 는 남습니다. C-2의 삭제 순서가 지켜지는 근거가 이것입니다.

나는 이 배열을 적은 적이 없습니다. 화살표 ①~⑩에서 자동으로 뽑혀 여기 적었습니다.

🔗 참고

dependencies 에는 직접 의존 외에 간접(전이) 의존까지 들어갑니다(association에 **aws_vpc.main** 이 함께 보이는 이유). 그래프 간선 수와 정확히 1:1은 아닙니다.

그리고 **aws_instance.web** 의 목록에 **aws_route_table_association.public** 은 없습니다. EC2는 라우트가 불기를 기다리지 않습니다 — 참조가 없으니까요. 오늘은 문제가 안 되지만, **user_data** 로 패키지를 받는다면 인터넷이 아직 없어서 실패하는 경쟁 상태가 됩니다. 그때 필요한 것이 **depends_on** 입니다(개념워크북 6번).

⑥ 기계용 JSON

```
terraform show -json | jq '.values.root_module.resources[] | {address, depends_on}'
terraform show -json > state.json      # 만들었으면 git status를 반드시 확인!
```

🚫 절대 금지

`state.json` 은 state와 같은 등급의 비밀입니다. `.gitignore` 가 막고 있지만 직접 확인하세요.

⑦ state pull

```
terraform state pull | jq -c '{version, serial, lineage, n:(.resources|length)}'
jq .serial terraform.tfstate      # 값이 그대로인지 확인
```

로컬 state에서는 `cat terraform.tfstate` 와 거의 같습니다. 이 명령이 진짜 의미를 갖는 건 Week3 S3 원격 backend입니다. 짝인 `state push` 는 위험해서 오늘 쓰지 않습니다.

[관찰 🏹] B-4 기록 — 이 문항 세트가 오늘의 정체성입니다

- `state list` 총 줄 수: _____ / 그중 `data.` 로 시작하는 줄: _____
- `state show aws_instance.web` 의 `private_ip` : _____ / `instance_state` : _____
- `tags` 의 키 개수: _____ / `tags_all` 의 키 개수: _____ / 차이가 나는 이유: _____
- `aws_instance.web` 의 `dependencies` 배열 전체: _____
- 그 배열이 화살표 지도의 몇 번에 해당하나: _____
- 그 배열에 `aws_route_table_association.public` 이 없는 이유: _____
- `aws_vpc.main` 의 `dependencies` : _____ (왜 그런가?)
- 최상위 `serial` 값: _____ (apply를 두 번 했는데 왜 이 숫자인가?)
- `mode` 가 "data" 인 항목 수: _____
- `terraform_version` : _____
- `state.json` 이 `git status` 에 보이는가: _____ (보이면 즉시 Discord)

[실습] B-5. drift 만들기 — 콘솔에서 손으로 바꿔보기

Week1의 [함정] 콘솔에서 손으로 고치면 을 직접 재현합니다.

AWS 콘솔 → VPC → Your VPCs → 내 VPC 선택 → Tags 탭 → Manage tags → Name 값 뒤에 `-BYHAND` 추가 → Save.



팁

콘솔이 헤매지면 CLI로도 됩니다.

```
# {본인-project_name} 을 terraform.tfvars 에 적은 값으로 바꿔 실행하세요
aws ec2 create-tags --region ap-northeast-2 \
  --resources "${terraform output -raw vpc_id}" \
  --tags Key=Name,Value={본인-project_name}-vpc-BYHAND
```

이제 두 명령의 차이를 봅시다.

```
terraform plan -refresh-only      # ① AWS가 바뀐 것만 보여줌. state를 건드리지 않는다
```

Note: Objects have changed outside of Terraform

Terraform detected the following changes made outside of Terraform since the last "terraform apply" which may have affected this plan:

```
# aws_vpc.main has been changed
~ resource "aws_vpc" "main" {
    id      = "vpc-0f9e8d7c6b5a4321f"
    ~ tags   = {
        ~ "Name" = "boaz26-w2-kdh1834-vpc" -> "boaz26-w2-kdh1834-vpc-BYHAND"
    }
}
```

```
terraform plan                    # ② Terraform이 무엇을 하겠다고 하는가
```

```
# aws_vpc.main will be updated in-place
~ resource "aws_vpc" "main" {
    id      = "vpc-0f9e8d7c6b5a4321f"
    ~ tags   = {
        ~ "Name" = "boaz26-w2-kdh1834-vpc-BYHAND" -> "boaz26-w2-kdh1834-vpc"
    }
    ~ tags_all = {
        ~ "Name" = "boaz26-w2-kdh1834-vpc-BYHAND" -> "boaz26-w2-kdh1834-vpc"
        # (4 unchanged elements hidden)
    }
}
```

Plan: 0 to add, 1 to change, 0 to destroy.

! 꼭 기억

화살표 방향을 보세요. 콘솔에서 바꾼 값 → 코드에 적힌 값 입니다. Terraform은 콘솔 변경을 반영하지 않고 되돌립니다.

코드가 정답이고 AWS는 그 사본입니다. 이게 IaC입니다. 그래서 IaC를 도입한 팀은 콘솔에서 손으로 고치는 걸 금지합니다 — 다음 apply에서 조용히 사라지니까요.

```
terraform apply # yes ← 복구가 별도 작업이 아니다. apply 자체가 복구다
```

📌 참고

Terraform이 못 잡는 drift도 있습니다. ① 코드에 안 적은 속성의 변경 ② 콘솔에서 아예 새로 만든 리소스(`import` 전에는 보이지 않음). 그래서 "state에 없으면 Terraform에게 존재하지 않는다"가 됩니다.

[관찰 🍷] B-5 기록

- `plan -refresh-only` 의 맨 위 Note 한 줄을 그대로: _____
- `terraform plan` 의 숫자: ____ to add, ____ to change, ____ to destroy
- Terraform이 제안한 방향 (콘솔 값 유지 / 코드 값으로 복구): _____
- `tags` 와 `tags_all` 이 둘 다 바뀐 이유: _____

[실습] B-6 (심화 ★). ~ vs -/+ 체험 — plan만, apply 금지

⚠ 주의

이 스텝에서는 절대 `apply` 를 누르지 마세요. plan만 보고 즉시 원복합니다. apply하면 EC2가 죽고 새로 만들어집니다 (시간 + 과금).

대조군 — `instance_type` 은 ~

`terraform.tfvars` 에서 `instance_type = "t2.micro"` 로 바꾸고 plan.

```
# aws_instance.web will be updated in-place
~ resource "aws_instance" "web" {
    id          = "i-0123456789abcdef0"
    ~ instance_type = "t3.micro" -> "t2.micro"
}
```

Plan: 0 to add, 1 to change, 0 to destroy.

EC2에는 `ModifyInstanceAttribute` API가 있어 중지 → 변경 → 시작으로 처리됩니다. 인스턴스 ID가 유지되므로 ~ 입니다. 하지만 다운타임은 있습니다 — ~ 가 곧 무해는 아닙니다.

본 실습 — subnet `cidr_block` 은 -/+ , 그리고 연쇄

`network.tf` 에서 `cidr_block = "10.0.1.0/24"` → `"10.0.2.0/24"` (한 글자) 후 plan.

Resource actions are indicated with the following symbols:

-/+ destroy and then create replacement

```
# aws_instance.web must be replaced
-/+ resource "aws_instance" "web" {
    ~ subnet_id = "subnet-0a1b2c3d4e5f67890" -> (known after apply) # forces replacement
}

# aws_route_table_association.public must be replaced
-/+ resource "aws_route_table_association" "public" {
    ~ subnet_id = "subnet-0a1b2c3d4e5f67890" -> (known after apply) # forces replacement
}

# aws_subnet.public must be replaced
-/+ resource "aws_subnet" "public" {
    ~ cidr_block = "10.0.1.0/24" -> "10.0.2.0/24" # forces replacement
}
```

Plan: 3 to add, 0 to change, 3 to destroy.

세 가지를 읽으세요.

- ① **# forces replacement** 주석의 위치 — 리소스 헤더가 아니라 원인이 된 속성 줄의 맨 끝입니다. 그 속성이 범인입니다.
- ② 연쇄 — 한 줄 고쳤는데 3개가 재생성됩니다. 서브넷이 죽으면 그 **id** 를 참조한 association과 EC2도 같이 죽습니다. **화살표** 지도가 곧 교체 전파 경로입니다.
- ③ **(known after apply)** 의 재등장 — "새 서브넷의 id를 아직 모른다 → 그걸 참조하는 인스턴스를 새로 만들어야 한다"는 인과가 이 한 줄에 다 들어 있습니다.

apply를 누르면 EC2가 죽고 퍼블릭 IP가 바뀝니다. 운영 중이라면 장애입니다.

반드시 원복

🚫 절대 금지

git checkout -- network.tf / git restore network.tf 를 쓰지 마세요. **practice/network.tf** 는 리포에 **TODO** 주석 스켈레톤 상태로 커밋돼 있습니다. git으로 되돌리면 cidr 한 글자가 아니라 **A-3·A-4**에서 채운 코드 전부가 사라집니다. 그러면 다음 **plan** 이 **No changes.** 가 아니라 **5 to destroy** 로 나옵니다.

손으로 되돌립니다.

```
# 1) network.tf 의 cidr_block 을 "10.0.2.0/24" -> "10.0.1.0/24" 로 직접 수정
# 2) terraform.tfvars 의 instance_type 을 "t3.micro" 로 직접 수정
git diff practice/network.tf # 스켈레톤과의 차이 = 내가 채운 코드. cidr가 10.0.1.0/24 인지 확인
terraform plan               # "No changes." 로 돌아왔는지 확인 ← 이 단계 필수
```

! 꼭 기억

No changes. 가 나오기 전에는 다음으로 넘어가지 마세요. 여기서 원복이 덜 되면 Block C의 destroy 숫자가 70이 아니게 됩니다.

[관찰 🏏] B-6 기록

- `instance_type` 변경은 ~ 인가 `-/+` 인가: _____
- `# forces replacement` 주석이 붙은 속성 이름: _____
- subnet cidr 변경 시 숫자: ____ to add, ____ to change, ____ to destroy
- 왜 3개인가 (화살표 번호로): _____
- apply 하지 않았음을 확인 — `terraform plan` 이 **No changes.** 인가: _____

[실습] B-7 (심화 ★★). cycle 만들어 보기

`compute.tf` 맨 아래에 SG 두 개를 서로 참조하도록 붙이고 `terraform plan`.

```
resource "aws_security_group" "a" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port      = 0
    to_port        = 0
    protocol       = "-1"
    security_groups = [aws_security_group.b.id]
  }
}

resource "aws_security_group" "b" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port      = 0
    to_port        = 0
    protocol       = "-1"
    security_groups = [aws_security_group.a.id]
  }
}
```

Error: Cycle: aws_security_group.a, aws_security_group.b

`plan` 단계에서 막히므로 AWS에는 아무 일도 일어나지 않습니다. 해결 방법은 규칙을

`aws_vpc_security_group_ingress_rule` 리소스로 빼는 것입니다(개념워크북 13번).

🚫 절대 금지

확인했으면 이 블록을 반드시 지우세요. Cycle 에러는 그래프를 만드는 단계에서 나므로 `plan` 뿐 아니라 `destroy` 도 실행되지 않습니다. 지우지 않고 Block C로 가면 destroy가 계속 실패하고 EC2가 계정에 남아 과금됩니다. 지운 뒤 `terraform plan` 이 `No changes.` 로 도는 것까지 확인하고 넘어가세요.

⚠️ 주의

인라인 `ingress` / `egress` 와 `aws_vpc_security_group_ingress_rule` 을 섞어 쓰지 마세요. 두 방식이 서로의 규칙을 지웁니다.

Block C. destroy + 잔존 점검 (11분) — 이걸 안 하면 오늘 실습은 실패입니다

[실습] C-0. 증빙 먼저 복사

! 꼭 기억

destroy하면 영원히 사라집니다. 과제 제출의 증빙이 이것뿐입니다. 지금 파일로 저장하세요.

```
# {github-id} 는 본인 GitHub ID 로 바꿔서 실행하세요 ( 셸이 자동으로 치환해주지 않습니다)
ID=본인-github-id
mkdir -p "../submissions/$ID"

terraform state list > "../submissions/$ID/state-list.txt"

# arn(계정번호 12자리) 과 public_ip 는 빼고 저장한다
terraform state show aws_instance.web \
  | grep -Ev 'arn|public_ip|public_dns' >> "../submissions/$ID/state-list.txt"

jq -r '.resources[] | "\(.type).\(.name) <- \(.instances[0].dependencies // [] | join(",
    "))"' \
  terraform.tfstate >> "../submissions/$ID/state-list.txt"
```

🚫 절대 금지

커밋 전에 **state-list.txt** 를 눈으로 한 번 읽으세요. 걸러야 하는 것은 두 가지입니다.

- 퍼블릭 IP — 22번이 열려 있던 서버 주소
- ARN 안의 계정번호 12자리 — `arn:aws:ec2:ap-northeast-2:123456789012:instance/i-...`

위 `grep -Ev` 가 대부분 걸러주지만, 남아 있으면 `x.x.x.x / <account-id>` 로 바꾸세요.

[실습] C-1. 삭제 순서를 먼저 예측하고 destroy plan 읽기

종이에 적어보세요. 화살표 지도를 뒤집으면 순서가 나옵니다.

[1단계]	_____		_____	
[2단계]	_____		_____	_____
[3단계]	_____			
[4단계]	_____			

이제 채점합니다.

```
terraform plan -destroy
```

```
Plan: 0 to add, 0 to change, 7 to destroy.
```

정답은 이렇습니다.


```
[1단계] aws_instance.web | aws_route_table_association.public  (앞사귀 - 아무도 이들을 참조하지 않는다)
[2단계] aws_security_group.web | aws_subnet.public | aws_route_table.public
[3단계] aws_internet_gateway.gw
[4단계] aws_vpc.main (마지막)
```

! 꼭 기억

삭제 순서는 apply 레벨을 그대로 뒤집은 것이 아닙니다. 의존 방향을 뒤집은 그래프에서 다시 계산합니다. 그래서 apply에서 레벨 3이던 `instance` 와 레벨 4이던 `association` 이 삭제에서는 같은 1단계가 됩니다 — 둘 다 아무도 참조하지 않는 앞사귀라서 먼저 지울 수 있기 때문입니다.

기준은 레벨 숫자가 아니라 "나를 참조하는 것이 아직 남아 있는가" 입니다. 그리고 그 판단의 근거가 B-4에서 본 state의 `dependencies` 배열입니다.

💡 팁

`terraform graph -type=plan-destroy` 로도 볼 수 있지만, 이 출력은 기본 graph와 달리 `"[root] aws_subnet.public (expand)"` 처럼 지저분합니다. 보고 싶으면 이렇게 걸러 쓰세요(선택).

```
terraform graph -type=plan-destroy \
| grep '\->' | grep -vE 'provider|output|root\[root\]' \
| sed 's/\[root\] //g; s/ (expand)//g; s/"//g; s/;/ /' | sort
```

[실습] C-2. destroy 실행

```
terraform destroy # yes
```

! 꼭 기억

Plan: 0 to add, 0 to change, 7 to destroy. 줄을 읽고 `yes` 를 치세요. 숫자가 7이 아니면 뭔가 남거나 덜 만들어진 것입니다.

⚠ 주의

EC2 삭제는 30~90초 걸립니다. Ctrl+C 금지. 인스턴스가 `terminated` 가 될 때까지 기다리고, 그 뒤에야 subnet·SG를 지울 수 있습니다(ENI가 정리돼야 하므로).

[확인] destroy 완료

```
aws_route_table_association.public: Destroying...
aws_instance.web: Destroying... [id=i-0123456789abcdef0]
aws_route_table_association.public: Destruction complete after 1s
aws_route_table.public: Destroying...
aws_route_table.public: Destruction complete after 1s
aws_instance.web: Still destroying... [30s elapsed]
aws_instance.web: Destruction complete after 41s
aws_security_group.web: Destroying...
aws_subnet.public: Destroying...
aws_subnet.public: Destruction complete after 1s
aws_security_group.web: Destruction complete after 1s
aws_internet_gateway.gw: Destroying...
aws_internet_gateway.gw: Destruction complete after 1s
aws_vpc.main: Destroying...
aws_vpc.main: Destruction complete after 1s

Destroy complete! Resources: 7 destroyed.
```

데이터 소스 2개는 이 숫자에 안 들어갑니다 — 만든 적이 없으니 지울 것도 없습니다.

[함정] destroy가 실패할 때

에러	원인	해결
<code>DependencyViolation: The subnet 'subnet-xxx' has dependencies and cannot be deleted</code>	ENI가 아직 살아 있음 / 콘솔에서 손으로 만든 것이 그 서브넷에 있음	1~2분 기다리고 <code>terraform destroy</code> 를 그냥 다시 실행. 대부분 이걸로 끝납니다
<code>DependencyViolation: Network vpc-xxx has some mapped public address(es)</code>	퍼블릭 IP를 쓰는 리소스가 남음	인스턴스 terminate 완료 대기 → destroy 재실행
<code>DependencyViolation: resource sg-xxx has a dependent object</code>	SG가 아직 ENI에 붙어 있음	동일. B-6에서 SG <code>name</code> / <code>description</code> 을 건드렸다면 먼저 원복
<code>Error acquiring the state lock</code>	다른 터미널의 명령이 돌고 있거나 이전 실행이 비정상 종료	끝날 때까지 대기. 진짜 죽은 락이면 <code>.terraform.tfstate.lock.info</code> 삭제 (Week3에서 <code>force-unlock</code> 다룸)
<code>Destroy complete! Resources: 6 destroyed.</code> (7이 아님)	중간 실패로 부분 삭제	<code>terraform state list</code> 로 남은 것 확인 → destroy 재실행



팁

destroy 는 몇 번을 돌려도 안전합니다. 이미 지워진 것은 건너뛰고 남은 것만 지웁니다. 실패했다고 당황해서 콘솔로 뛰어가지 말고, 일단 다시 실행하세요.

[확인] C-3. state가 진짜 비었나 — 두 겹으로 확인

```
terraform state list          # 완전히 빈 출력이어야 한다
jq '.resources | length' terraform.tfstate # 0 이어야 한다
jq '.serial' terraform.tfstate      # A-6 때보다 커져 있다
```

! 꼭 기억

state list 가 완전히 비어야 성공입니다. 데이터 소스까지 함께 사라집니다 — 관리할 리소스가 없으면 읽어들 이유도 없습니다.

그런데 **terraform.tfstate** 파일 자체는 남아 있고 **serial** 은 오히려 커졌습니다. 장부는 남고 내용만 비었습니다. destroy도 하나의 저장(save)이니깐요.

[관찰 📖] C-2·C-3 기록

- 가장 먼저 **Destroying...** 이 뜬 리소스: _____ / 그것이 **일사귀인** 이유: _____
- 가장 오래 걸린 삭제와 초: _____
- 예측(C-1)이 틀린 지점: _____
- destroy 후 **state list** 줄 수: _____
- resources** 배열 길이: _____ / **serial**: _____ → 왜 늘었나: _____

[실습] C-4. 계정 잔존 점검 — "state가 비었다 ≠ 계정이 비었다"

콘솔 우측 상단이 **서울(ap-northeast-2)** 인지 먼저 확인하세요. 리전이 다르면 아무것도 안 보이는 게 당연합니다.

확인할 것	콘솔 경로	정상
EC2 인스턴스	EC2 → Instances	terminated 만 (running / stopped 는 사고)
EBS 볼륨	EC2 → Volumes	available 상태인 볼륨 0개
Elastic IP	EC2 → Elastic IPs	0개
NAT Gateway	VPC → NAT Gateways	0개
ENI	VPC → Network Interfaces	내 VPC의 것 없음
VPC	VPC → Your VPCs	내 VPC 없음

CLI로 한 번에 확인하는 스크립트가 리포에 있습니다.

```
bash ../scripts/check-leftover.sh
```

전부 0이어야 정상

! 꼭 기억

CP3 — 빈 `state list` 출력 + 아래 체크 결과를 Discord에 올려주세요. `[ ] EC2 terminated [ ] Volume 0`
`[ ] EIP 0 [ ] NAT 0 [ ] VPC 없음`

8명 전원 CP3 확인 전에는 세션을 끝내지 않습니다.

[실습] C-5. 커밋 사고 방지

```
git status
```

파일	이래야 정상
<code>terraform.tfvars</code>	안 보임 (<code>.gitignore</code>) — 내 공인 IP가 들어 있다
<code>terraform.tfstate</code>	안 보임
<code>state.json</code>	안 보임 ← B-4에서 만든 것
<code>*.pem</code>	안 보임 (오늘 만들지 않았지만)
<code>.terraform/</code>	안 보임
<code>practice/network.tf</code> , <code>compute.tf</code> , <code>outputs.tf</code>	보임(수정됨) → 여기서 커밋하지 말고 아래 절차대로 <code>submissions/{id}/</code> 로 복사
<code>.terraform.lock.hcl</code>	안 보임이 정상 — 이미 커밋된 파일이고 내용이 안 바뀌었기 때문

🚫 절대 금지

`terraform.tfvars` / `terraform.tfstate` / `state.json` 이 목록에 보인다면 커밋하지 말고 멈추세요.
`.gitignore` 가 안 먹은 상태입니다. Discord에 알려주세요.

세션 후 — 실습 결과 제출 (브랜치 → PR)

세션 중에는 그냥 `practice/` 에서 실습하면 됩니다. 끝나고 아래를 하세요.

```
ID=본인-github-id # {github-id} 는 셸이 치환해주지 않습니다. 변수로 잡고 시작하세요

git switch main && git pull
git switch -c "week2/$ID"

mkdir -p "submissions/$ID"
cp practice/*.tf practice/example.tfvars "submissions/$ID/"
cp practice/.terraform.lock.hcl "submissions/$ID/"
# C-0에서 저장한 state-list.txt 는 이미 그 폴더에 있습니다 (IP·계정번호 마스킹 확인!)
# 관찰 🍷 답안을 submissions/$ID/observations.md 로 정리

git add "submissions/$ID"
git status # tfvars / tfstate / state.json 안 올라갔는지!
git commit -m "week2: $ID 실습 제출"
git push -u origin "week2/$ID"
```

복사가 끝나면 `practice/` 는 되돌려 둡니다. 다음 주에 이 리포를 다시 열 때 깔끔한 상태가 됩니다.

```
git restore practice/ # 복사를 끝낸 뒤에만! 스켈레톤으로 되돌아갑니다
```

그다음 GitHub에서 `main` 대상 PR을 올리면 강사가 리뷰 후 머지합니다.

! 꼭 기억

제출은 `submissions/{github-id}/` 본인 폴더에. `practice/` 를 직접 고쳐서 올리면 머지하는 순간 다음 사람이 풀 # TODO 가 사라집니다. 폴더를 나눠야 8명 PRO이 전부 머지됩니다.

체크포인트 (Definition of Done)

`terraform apply` 두 번 성공 (5 added → 2 added)

`state list` 에 9줄 (리소스 7 + 데이터 소스 2) 확인

`state show aws_instance.web` 으로 `private_ip` · `public_ip` · `instance_state` 확인

`dependencies` 배열을 열어 화살표 지도와 대조 (오늘의 핵심)

drift 만들고 `plan` 이 되돌리자고 제안하는 것 확인 (B-5)

`terraform destroy` 완료 (7 destroyed) & `state list` 빈 출력

콘솔/스크립트로 EC2·Volume·EIP·NAT·VPC 잔존 0 확인 (C-4)

`git status` 로 `*.tfvars` / `*.tfstate` / `state.json` 안 올라갔는지 확인

오늘의 명령 요약

화살표 지도 순서 그대로입니다.

```
curl -4 ifconfig.me # A-2 내 공인 IP
cp example.tfvars terraform.tfvars # A-2 값 채우기
# A-3~A-4 화살표 ①~⑦ 굿기

terraform fmt && terraform init # A-5
terraform validate # A-5 init 이후에
terraform plan # A-5 "5 to add"
terraform apply # A-6 네트워크 5개
terraform state list # A-6 7줄
terraform graph | grep '\->' # A-7 그래프를 텍스트로
# B-1 화살표 ⑧~⑩ 굿기

terraform plan # B-2 "2 to add"
terraform apply # B-3 EC2 (30~60초, Ctrl+C 금지)
terraform state show aws_instance.web # B-4 속성 파보기
jq 'keys' terraform.tfstate # B-4 최상위 키 7개
jq -r '.resources[] | .instances[0].dependencies' terraform.tfstate # B-4 ★
terraform plan -refresh-only # B-5 drift만 보기
terraform plan -destroy # C-1 삭제 순서 예측 채점
terraform graph -type=plan-destroy # C-1 역방향 그래프
terraform destroy # C-2 7개 삭제 - 필수!
terraform state list # C-3 빈 출력이어야 함
bash ../scripts/check-leftover.sh # C-4 계정 잔존 0 확인
```

다음 주 예고

Week 3. Remote Backend & State 잠금 — 오늘 뜯어본 `terraform.tfstate` 가 그대로 문제가 됩니다.

오늘 본 것	Week3에서
장부가 내 노트북에만 있다	팀원 8명은 이 장부를 못 본다 → S3 원격 backend
<code>serial</code> 이 저장마다 +1	두 명이 동시에 apply하면 충돌 → 잠금(lock)
state가 평문 JSON	원격에 두려면 암호화·접근 제어
<code>lineage</code> 가 혈통을 보증	잘못된 장부를 밀어 넣는 사고를 어떻게 막나

예습: 원격 state가 왜 필요한지, `backend "s3"` 블록, `terraform init -migrate-state`

참고 문서

- [Create Resource Dependencies](#) — 이 실습의 뼈대

- [Build and Use a Local Module](#) — 뒤 주차 연습용
- [Manage Resource Drift](#) — B-5의 원본
- [terraform graph](#) · [terraform state](#) · [terraform show](#)
- [aws_instance](#) · [aws_subnet](#) · [aws_route_table](#) · [aws_security_group](#)
- [aws_ami \(data source\)](#) · [aws_availability_zones](#)
- [Amazon EC2 요금](#) · [퍼블릭 IPv4 주소 요금](#)

이 문서의 plan 출력·AMI ID·AZ·비용은 **서울 리전**, **AWS provider 6.57.1**, **Terraform 1.15.8** 에서 실제로 조화·검증한 값입니다.
인스턴스 ID나 IP는 예시로 바꿔 적었습니다.

! 꼭 기억

끝났으면 **terraform destroy** 를 정말로 했는지, 그리고 콘솔에 EC2가 **terminated** 인지 한 번 더 확인하세요. 안 지운 EC2는 한 달 뒤 약 2만원의 청구서로 돌아옵니다.